

ROS-based SLAM and Navigation for a Gazebo-Simulated Autonomous Quadrotor

Y. Alborzi^a, B. Safari Jalal^a and E. Najafi^{a,b}

^aFaculty of Mechanical Engineering, K.N. Toosi University of Technology, Tehran, Iran

^bMechatronics Department, Fontys University of Applied Sciences, Eindhoven, The Netherlands

Emails: yusefalborzi@email.kntu.ac.ir; behzad.safari@email.kntu.ac.ir; e.najafi@fontys.nl

Abstract—Effective robot navigation is crucial for the proper implementation of a mobile robot. To navigate a robot in an indoor or outdoor environment, a map of the surrounding obstacles is required. In order to acquire this map, the robot uses sensors to collect information about its surroundings and its own location. This paper presents a robotic operating system based on autonomous simultaneous localization and mapping (SLAM), and robot navigation implementation of a Parrot AR.Drone 2.0 quadrotor, which is equipped with a laser scanner and inertial measurement unit. For autonomous mapping, SLAM along with frontier type exploration has been used to acquire a full 2D map of the environment in Gazebo. The A* search algorithm is utilized for the global planner, while the dynamic window approach has been implemented for the local planner. The Gazebo-simulated results validate the competent performance of the underlying structure of both the proposed SLAM as well as the provided navigation algorithm.

I. INTRODUCTION

The impact of robots on our lives has become indisputable. Nowadays, there is a robot for almost any task, including the use of robots in surgery, agriculture and industry. One type of robot getting more and more attention, is unmanned aerial vehicle (UAV). These are aircraft without a pilot on board and can be classified depending on their flying mechanisms. Generally, they are characterized as fixed-wing, flapping-wing or rotorcrafts. While fixed-wing UAVs are preferred for fast data collection in large fields, rotorcrafts are used in conditions where more precise data acquisition and actions are required or in the case of small, constrained and heterogeneous environments; Because they can hover and are easier to control.

The UAVs consist of motors to enable movement, on-board computers to control these motors and finally, radio transmitters for remote controllers or processors to navigate autonomously. They are used in both inside and outside environments and unlike ground robots, they are not challenged by rough terrains, which enables a wide range of maneuverability [1]. In addition, a top plane view allows a UAV to cover areas that ground robots are not capable of doing. These features have made UAVs popular and they are employed for more applications. Although initially driven by the military section, civilian applications have been growing at a faster rate in possible application for surveillance, con-

struction, photography and agriculture. The growing interest in civilian or commercial UAVs has lead to improvements in their functionality and ease of use and has made them more appealing to a wider range of customers. However, regardless of these improvements, the battery problems still persist. Because of the limited payload, most UAVs can not carry large batteries and therefore have a limited flight time. With this in mind, navigating the drone to desired goals efficiently has become an important task [2].

In order to navigate the drone through the environment safely, at first, Information about the surroundings of the robot is required. The robot needs to know where the obstacles are and where it is with respect to the obstacles. Acquiring information about surrounding obstacles is called “Mapping” and finding the robot’s state is referred to as “Localization”. A robot navigating without mapping and localization is similar to a person moving in a completely dark room. And just as a human has eyes to observe its surroundings a robot requires tools to get feedback from neighboring obstacles. For this purpose different sensors are used to find the distance between the robot and the obstacles. These include laser scanners, ultrasonic sensors and cameras, which naturally require different computation approaches. Furthermore, the process of mapping the environment can be accomplished manually or autonomously. The autonomous mapping process is labeled “Exploration”, in which the robot moves autonomously through its workspace without running into obstacles while simultaneously and progressively creating a map of it. After the map is acquired, the robot can use it to move safely inside its configuration space.

This paper illustrates the deployment and assessment of an autonomous quadrotor robot using the robotic operating system (ROS) and the Gazebo simulation tool. The robot autonomously explores the simulated world and creates a 2D map of it. Then, an efficient navigation technique is implemented to move the robot towards arbitrary desired goals. The results of the simulation are presented and the utilized communication system structure is explained.

The paper is organized as follows. A short review of different mapping, exploration and navigation algorithms is given in Section II. Section III studies the implemented quadrotor system structure. Section IV displays the results and finally, Section V concludes the paper.

II. RELATED WORKS

This section mainly reviews the related literature in mobile robot mapping, exploration and navigation. The task of localization is crucial for an efficient navigation system and because pure reliance on odometry sensors such as IMU is prone to error, Kalman filters are used to integrate different sensors for a successful localization [3]. Furthermore, the robot uses this information of its own state and the information of the sensors to create a map of the environment simultaneously. This process is called simultaneous localization and mapping (SLAM) [4]. Over the years, several SLAM techniques have been developed. Works such as EKF-SLAM [5] use Kalman filters to linearize the plant and the measurement model to compensate for the assumed non-linear Gaussian noise of the system. However, Kalman filter approaches are prone to error propagation which can cause inconsistency in SLAM.

On the other hand, particle filter approaches do not pose such challenges. Unlike the linear Kalman filter estimators, particle filters do not make assumptions about the noise in the system. FastSLAM [6] and DP-SLAM [7] are examples of these methods. Particle filter approaches take into account a large number of samples to estimate the system state by representing the state variables with a probability density function. Moreover, there have also been approaches that use images to perform SLAM, which are called visual SLAM. The Monocular SLAM approach in [8] is an example of this method, which only uses a single camera to create large-scale consistent maps.

Exploration is the task of commanding the robot to visit previously unseen areas with the goal of increasing its knowledge of the workplace and it stands at the core of autonomous mapping. There have been several approaches to deal with this problem. Topological methods have been developed that describe the environment in graphs, meaning that they do not result in metric measurements such as the work of [9]. Furthermore, works such as [10], [11] use potential field methods which can find suitable trajectories by calculating the gradient descent of assigned potential values. Moreover, frontier-based exploration was introduced by [12] in which the robot identifies points between the free-space and the unknown space and moves towards them to expand the map. Finally, information gain exploration not only takes into account mapping the environment but also tries to optimize this process with maximizing information gain in each step by estimating the number of expected frontier cells. Such an algorithm can be seen in the work of [13], [14].

One of the most important function of robot navigation is finding a safe path from the initial position to the desired position. The task of robot path planning consists of two steps: Environmental modeling and path search algorithm. The environmental modeling is based on the acquired map of the working space and is a feature information map of the environment. The approaches for this task include the cell decomposition, probabilistic road map, Artificial Potential Field (APF) and the rolling window approach. In the cell

decomposition approach the environment is divided into non-overlapping grids classified as empty or occupied. Connectivity between the empty cells, which do not contain obstacles, creates a path from the start position to the goal position. There have been different variations of this approach such as the exact cell decomposition [15], approximate cell decomposition [16] and adaptive cell decomposition [17], [18]. Moreover, in the road map approaches the space between the graph nodes are represented by curves in which the planner will try to find the most ideal path. The visibility graph [19] and the voronoi diagram [20] are examples of this method. Furthermore, APF methods [21] assign potential values to goal positions and obstacles, and the total potential leads to an imaginary force which moves the robot towards the goal and away from the obstacles. Applications of this method are discussed in works such as [22]–[24].

The path search stage consists of finding the optimum paths created in the previous environmental modeling stage. Search algorithms for this task include simpler approaches such as Dijkstra [25], A* [26] and D* [27] algorithms and also artificial intelligence methods such as genetic algorithms, neural networks and particle swarm optimization. The Dijkstra algorithm can find the shortest path between two points by computing the minimum cost required to reach every single node that encircles the start node and expanding its path towards further nodes until it reaches the goal node. The A* algorithm is based on the Dijkstra algorithm, and it also has a heuristic function that takes into account the position of the goal node as well, which makes this algorithm more efficient in finding the shortest path towards the goal. This heuristic function is usually the Euclidean distance between each node and the goal node.

III. SYSTEM STRUCTURE

To assess the efficiency of the quadrotor navigation system, this study contains a simulation of a quadrotor implemented in the robotic operating system, called ROS. The platform ROS is an open source software with innovative communication modules and tremendous package management tools. Additionally, it is supported by a large community of developers consistently providing new packages, and it also supports convenient tools for visualization and simulation such as Gazebo and Rviz. This section demonstrates the simulation of a Parrot AR.Drone 2.0 quadrotor in ROS. The utilized packages and underlying system communication structure are described in detail.

A. Model Specification

In order to simulate a robot in ROS, the physical specifications and the dynamics of the robot must be defined. The “hector-quadrotor” package is a powerful package for simulating and controlling a quadrotor in ROS and Gazebo environments, provided by the “Team Hector” [28]. Since this package does not use Parrot AR-Drone 2.0 as its simulation drone, its model was changed by the URDF drone model file provided by the “tum-simulator” package. This drone is equipped with an IMU and a 2D laser rangefinder (Hokuyo



Fig. 1. Simulated model of the Parrot AR.Drone 2.0 in Gazebo

UTM-30LX), which can be used for the mapping and localization processes, as depicted in Fig. 1).

B. Localization and Mapping

As mentioned before, SLAM is used to map the environment. In this study, The “gmapping” and “amcl” packages are used for the SLAM process. The “gmapping” package enables us to implement OpenSlam’s Gmapping algorithm in the ROS environment. It can learn and create occupancy grid maps using laser range data in a highly efficient manner based on Rao-Blackwellized particle filters [29].

One of the most efficient and popular probabilistic algorithms for local and global localization of a mobile robot, is the Monte-Carlo localization [30]. It uses a particle filter for estimating and tracking the current position of the robot and its orientation in a known map. The “amcl” package implements the adaptive (or KLD-sampling) Monte-Carlo localization approach. To adapt these packages to our usage, the files provided by the “gmapping” and “amcl” packages were modified to correspond to the Parrot AR.Drone 2.0 model.

C. Exploration

Initially, the robot knows nothing about its environment except the area around its current position covered by its laser rangefinder, as shown in Fig. 2. By using the implemented “slam-gmapping” package, the robot starts to map the surrounding area. Then, to extend the created map, the robot should start moving towards the unknown area, until the workspace is completely mapped. This process is done using the “explore-lite” package, which is a frontier-based exploration package [31]. A frontier is an area between an unoccupied cell, which is already mapped, and unknown cells in the 2D occupancy grid map [12]. The “explore-lite” package can use the costmaps provided by either the mapping algorithm or the “move-base” package instead of constructing its own costmap. This makes it more efficient and lightweight. Fig. 3 shows detected frontier points of the constructed map.

D. Navigation

First, a global planner computes an efficient path towards an arbitrary goal point regarding the static obstacles within the constructed map, using the A* algorithm. Then, the

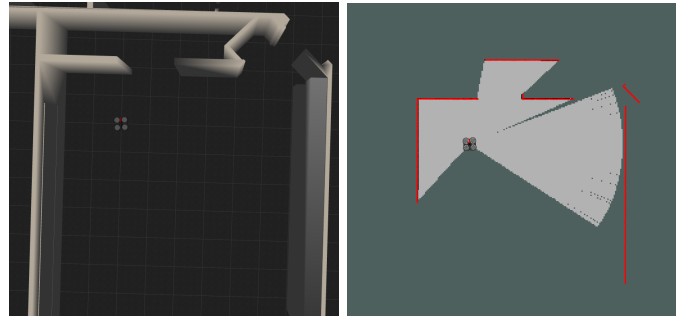


Fig. 2. Initial state of the robot in the simulated workspace (left), beginning of the mapping process in the unknown workspace (right)

local planner computes appropriate and efficient velocity commands for the drone, so it can follow its path towards the goal while avoiding the dynamic obstacles. The “dwa-local-planner” package handles this process by the Dynamic Window Approach (DWA) method [32]. Like the earlier mentioned packages, the “move-base” package files were also modified accordingly.

To simplify the control and navigation of the quadrotor, the materials used for SLAM, motion planning and robot control, were all two dimensional. The aforementioned packages were mostly developed for ground robots and therefore unable to control the robot’s altitude. To address this problem, a node was written to elevate the drone to a fixed altitude above the ground at the beginning of the motion. The PID controller makes sure that the drone will obey the 2D velocity commands provided by the local planner while keeping the drone at the fixed altitude.

E. Communication System Structure

The underlying relationships between the various nodes and packages has been demonstrated in Fig. 4. A number of the more critical parts of this graph are described below:

- At first, the “move_up” node elevates the robot to a fixed altitude by publishing to the “/action/pose/goal” topic.
- The “/slam_gmapping” node creates the map by publishing the “/map” topic, using “/scan” and “/tf” topics, which

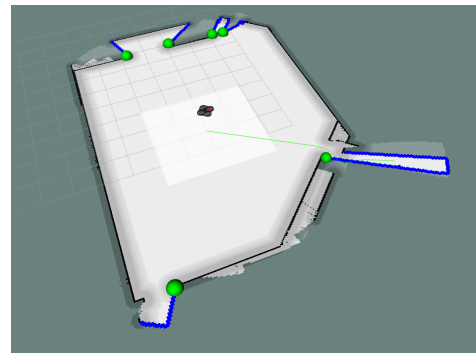


Fig. 3. Frontier points detected by the implemented algorithm (green points)

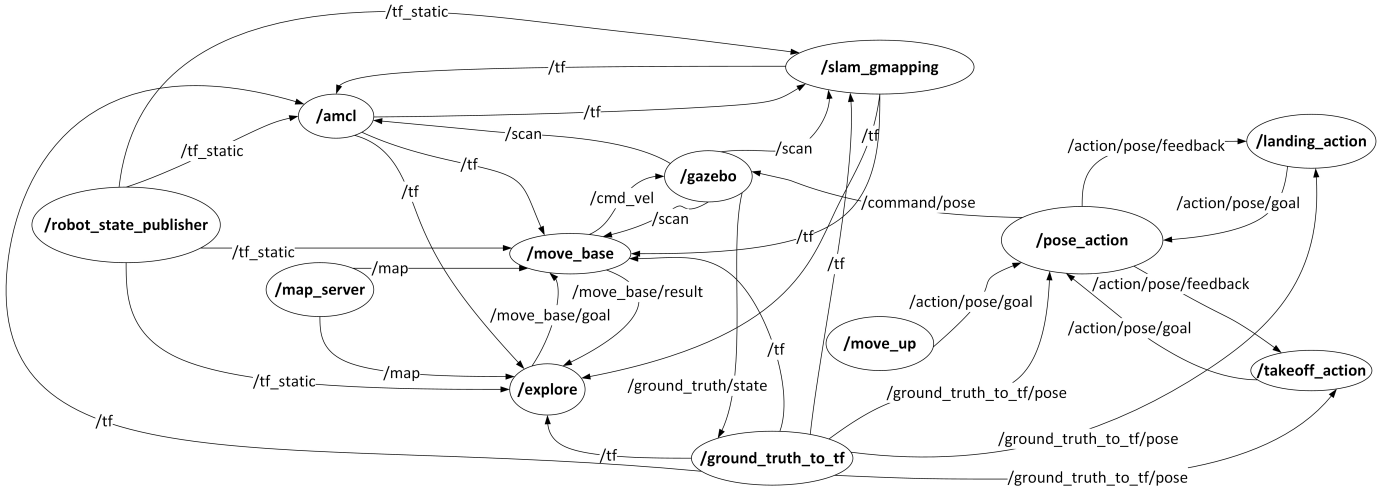


Fig. 4. Underlying communications between different nodes in ROS

are the rangefinder sensor data and robot's current pose respectively.

- The “amcl” node localizes the drone by subscribing to “/scan” and “/tf” topics, and publishing “/tf” topic. This node also provides the particle clouds, which can be visualized by Rviz.
- The “/explore” node uses the constructed map and robot's current pose provided by the “/map” and “/tf” topics respectively, to send 2D goal poses via the “/move_base/goal” topic.
- The “/move_base” node subscribes to the “/map” and “/scan” topics provided by the “/slam_gmapping” and “/gazebo” nodes, which are the constructed map and laser data respectively, to generate global and local costmaps. It also uses the “/tf” topic which indicates the robot's current pose, and “/move_base/goal” topic published by “/explore” node, to compute a path towards the goal and publish to the “/cmd_vel” topic, which contains the appropriate velocity commands for drone navigation. The user can also send goal poses manually by the “2D Nav Goal” button provided by Rviz.

IV. SIMULATION RESULTS

Initially, the “amcl” package was able to approximately localize the quadrotor's pose. As the robot moves around, it gathers more and more information regarding its surrounding environment and thus, the uncertainty of the localization system decreases. Fig. 5 visualizes this phenomenon by displaying the condensation of the particle clouds.

The implemented SLAM and exploration packages were tested on the “willow garage” world. Frontiers of the constructed map were successfully detected by the “explore-lite” package and a large section of the complicated simulated environment was successfully mapped autonomously. The result is displayed in Fig. 6. Moreover, the implemented 2D navigation system can generate safe trajectories to navigate the drone to

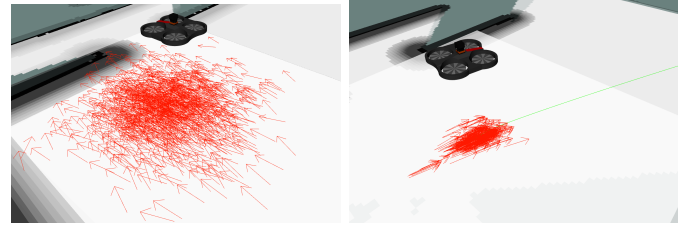


Fig. 5. Visualized particle clouds condensation process of the “amcl” package

its desired goal in the workspace. Fig. 7 shows the constructed path towards an arbitrary goal point using the global planner.

V. CONCLUSION

In this work, an autonomous system for quadrotor exploration and navigation was implemented on a simulated Parrot AR.Drone 2.0. Additionally, an accurate SLAM algorithm was used to create maps from the environment. The ROS simulator was utilized to model the quadrotor and implement the required systems. Furthermore, the Gazebo interface was used as the simulation environment. The adopted packages were modified to navigate the quadrotor at a fixed height in



Fig. 6. Simulated environment in Gazebo (left), constructed map with frontier exploration (right)

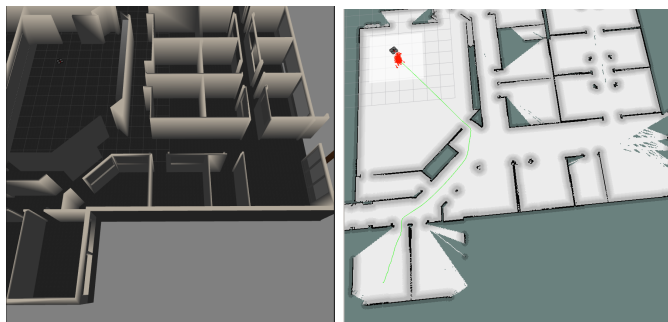


Fig. 7. Generated path by the global planner of the navigation stack

a 2D manner with the help of a PID controller. Finally, the results of the exploration, mapping and navigation algorithms were presented. For future works, the navigation system can be improved with a 3D approach, enabling it to generate more complex trajectories for the quadrotor. Moreover, real world implementations should be evaluated to further assess the efficiency of the simulated system.

REFERENCES

- [1] E. Najafi, T. Laugs, P. N. Rubio, and S. Alers, "An implementation of AutomationML for an Industry 4.0 case study: RoboCup@Work," in *Proceedings of the International Conference on Research and Education in Mechatronics*, 2019, pp. 1–6.
- [2] A. Taleizadeh, E. Najafi, H. N. Pishkenari, and A. Alasty, "Deployment of model-based design approach for a mini-quadcopter," in *Proceedings of the 7th International Conference on Robotics and Mechatronics*, 2019, pp. 291–296.
- [3] L. F. De Melo and J. F. M. Junior, "Trajectory planning for non-holonomic mobile robot using extended Kalman filter," *Mathematical problems in engineering*, vol. 2010, 2010.
- [4] T. Bailey and H. Durrant-Whyte, "Simultaneous localization and mapping (SLAM): Part II," *IEEE robotics & automation magazine*, vol. 13, no. 3, pp. 108–117, 2006.
- [5] J. E. Guivant and E. M. Nebot, "Optimization of the simultaneous localization and map-building algorithm for real-time implementation," *IEEE transactions on robotics and automation*, vol. 17, no. 3, pp. 242–257, 2001.
- [6] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit, "FastSLAM 2.0: an improved particle filtering algorithm for simultaneous localization and mapping that provably converges," in *IJCAI*, 2003, pp. 1151–1156.
- [7] A. I. Eliazar and R. Parr, "DP-SLAM 2.0," in *Proceedings of the IEEE International Conference on Robotics and Automation*, vol. 2, 2004, pp. 1314–1320.
- [8] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: large-scale direct monocular SLAM," in *Proceedings of the European conference on computer vision*, 2014, pp. 834–849.
- [9] G. Dudek, M. Jenkin, E. Milios, and D. Wilkes, "Robotic exploration as graph construction," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 6, pp. 859–865, 1991.
- [10] E. P. e Silva Jr, P. M. Engel, M. Trevisan, and M. A. Idiart, "Exploration method using harmonic functions," *Robotics and Autonomous Systems*, vol. 40, no. 1, pp. 25–42, 2002.
- [11] E. Najafi, G. A. Lopes, S. P. Nagesh Rao, and R. Babuska, "Rapid learning in sequential composition control," in *Proceedings of the 53rd IEEE International Conference on Decision and Control*, 2014, pp. 5171–5176.
- [12] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997, pp. 146–151.
- [13] L. Freda, G. Oriolo, and F. Vecchioli, "Sensor-based exploration for general robotic systems," 2008, pp. 2157–2164.
- [14] G. A. Lopes, E. Najafi, S. P. Nagesh Rao, and R. Babuska, "Learning complex behaviors via sequential composition and passivity-based control," in *Handling Uncertainty and Networked Structure in Robot Control*, L. Busoniu and L. Tamas Eds. Springer, 2015, pp. 53–74.
- [15] J. T. Schwartz and M. Sharir, "On the 'piano movers'" problem I. the case of a two-dimensional rigid polygonal body moving amidst polygonal barriers," *Communications on pure and applied mathematics*, vol. 36, no. 3, pp. 345–398, 1983.
- [16] D. J. Zhu and J. . Latombe, "New heuristic algorithms for efficient hierarchical path planning," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 1, pp. 9–20, 1991.
- [17] H. Samet, "An overview of quadrees, octrees, and related hierarchical data structures," in *Theoretical Foundations of Computer Graphics and CAD*. Springer, 1988, pp. 51–68.
- [18] E. Najafi and G. A. Lopes, "Towards cooperative sequential composition control," in *Proceedings of the IEEE 55th Conference on Decision and Control*, 2016, pp. 4758–4763.
- [19] L. Lulu and A. Elnagar, "A comparative study between visibility-based roadmap path planning algorithms," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2005, pp. 3263–3268.
- [20] H. M. Choset, S. Hutchinson, K. M. Lynch, G. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun, *Principles of robot motion: theory, algorithms, and implementation*. MIT press, 2005.
- [21] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Proceedings of the IEEE International Conference on Robotics and Automation*, vol. 2, 1985, pp. 500–505.
- [22] J.-O. Kim and P. Khosla, "Real-time obstacle avoidance using harmonic potential functions," 1992.
- [23] E. Najafi, A. Shah, and G. A. Lopes, "Robot contact language for manipulation planning," *IEEE/ASME Transactions on Mechatronics*, vol. 23, no. 3, pp. 1171–1181, 2018.
- [24] E. Najafi, R. Babuska, and G. A. Lopes, "An application of sequential composition control to cooperative systems," in *Proceedings of the 10th International Workshop on Robot Motion and Control*, 2015, pp. 15–20.
- [25] E. W. Dijkstra *et al.*, "A note on two problems in connexion with graphs," *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [26] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [27] A. Stentz, "Optimal and efficient path planning for partially known environments," in *Intelligent unmanned ground vehicles*. Springer, 1997, pp. 203–220.
- [28] J. Meyer, A. Sendobry, S. Kohlbrecher, U. Klingauf, and O. Von Stryk, "Comprehensive simulation of quadrotor UAVs using ROS and Gazebo," in *Proceedings of the International conference on simulation, modeling, and programming for autonomous robots*, 2012, pp. 400–411.
- [29] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [30] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005.
- [31] J. Hörner, "Map-merging for multi-robot system," 2016.
- [32] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.